

SIH26066 OCEANEMBED — COMPLETE TECHNICAL RESEARCH & BUILD BIBLE

Source discipline used throughout: every claim is tagged [A] officially stated by SIH, [B] inferred from the PS, [C] proposed by us, [D] from research literature, or [E] our engineering recommendation. Where something could not be verified, it is stated explicitly rather than guessed.

Executive Summary

OceanEmbed asks a six-student team to build a deep-learning pipeline that ingests daily, 0.25°-resolution **surface** satellite/ocean fields (SST, SSS, SSH/SLA, surface currents U/V, surface winds U/V) over the **North Indian Ocean (5°N-30°N, 45°E-105°E)** and reconstructs the **subsurface temperature profile at 15 standard depths (0-1000 m)** [A]. The training target is the **GLORYS global ocean reanalysis**; independent validation uses **gridded ARGO** via the INCOIS Live Access Server [A]. The core technical challenge is not "can a neural net fit this" — several published papers already do variants of this (Part 16) — it is **doing the data engineering correctly, avoiding leakage, validating honestly, and being able to defend every choice**. This document is written so a team with zero oceanography or scientific-ML background can go from nothing to a defensible working prototype.

One caveat up front [A/gap]: the mirrored problem-statement page we retrieved shows a "Training Input Datasets" table placeholder that did not render ("Insert table here"), and a "View attached dataset" link that requires the logged-in SIH portal to open. **Your very first action must be to log into sih.gov.in with your registered team credentials and open the actual attached dataset table/files for SIH26066** — do not assume our Part 6/7 dataset recommendations are the official list until you've confirmed against the real attachment.

1. Official Problem Statement (extracted verbatim in structure, paraphrased in wording)

Field	Value	Source
PS ID	SIH26066	[A]
Title	OceanEmbed - Satellite Embedding-Based Deep Learning Framework for Reconstruction of Subsurface Ocean Temperature from Surface Satellite Observations	[A]
Ministry	Ministry of Earth Sciences (MoES)	[A]
Department	Indian National Centre for Ocean Information Services (INCOIS), Ocean Valley	[A]
Category	Software	[A]
Theme	Disaster Management (portal also tags it "Space Technology" in one catalogue mirror — discrepancy noted)	[A]/discrepancy
Deadline (idea submission on portal)	30 September 2026	[A]
Geographic domain	North Indian Ocean, 5°N-30°N, 45°E-105°E	[A]
Spatial resolution	0.25° × 0.25°	[A]
Temporal resolution	Daily	[A]
Input (surface) variables	SST, SSS, SSH/SLA, surface currents (U,V), surface winds (U,V)	[A]
Target variable	Temperature (not salinity) at depth	[A]
Depth levels	0, 5, 10, 20, 30, 50, 75, 100, 125, 150, 200, 300, 500, 700, 1000 m (15 levels)	[A]
Training target dataset	GLORYS Global Ocean Reanalysis, https://doi.org/10.48670/moi-00021 , variable: Temperature	[A]
Independent validation dataset	Gridded ARGO via INCOIS Live Access Server (LAS)	[A]
Regridding permission	If a dataset isn't natively at 0.25°/daily, the team may pick an openly available product and interpolate/regrid	[A]
Expected solution components	(1) preprocessing pipeline, (2) satellite embedding engine, (3) DL reconstruction model, (4) standardized 0.25°/daily output, (5) validation framework using independent ARGO, (6) working PoC demo over Bay of Bengal / Arabian Sea	[A]
Suggested architecture families	CNN, Vision Transformer, Autoencoder, GNN, attention-based hybrids — explicitly offered as options, not mandates	[A]
Input dataset table	Placeholder unresolved on the mirror we accessed; check sih.gov.in directly	gap

Requirement → interpretation table

Requirement	Exact SIH requirement	Interpretation	Engineering implication
"Satellite embedding"	Generate "compact satellite embeddings" from surface fields using DL	[B] They want an explicit learned latent representation stage, not just an end-to-end black-box regressor	Architecture must have a visible encoder → latent vector → decoder/head split you can point to in the demo
"Reconstruct temperature at standard depths"	15 fixed depths, not a continuous function	[B] Treat depth as a discrete output dimension (15-channel output) or as an auxiliary conditioning variable	Output tensor shape choice affects loss weighting and architecture (Part 12)
"Validation framework using independent ARGO"	ARGO used for validation, GLORYS for training	[B] SIH wants you to demonstrate the model isn't just parroting GLORYS	You must show ARGO-based skill metrics as the headline number, not GLORYS-internal validation
"PoC over Bay of Bengal / Arabian Sea"	Demo scope narrower than full domain	[C] Full 5–30°N/45–105°E training is fine, but demo can focus on one sub-basin for clarity	Keep a fast "demo mode" that loads a small spatial/temporal slice
Regridding permission	Explicit permission to interpolate if native resolution differs	[A] This anticipates that SSS/currents/winds will NOT natively be 0.25° daily	Build a generic regridding module early; treat it as core deliverable, not a footnote

2. The Problem, Explained From Zero

2.1 What is ocean temperature, and why does it change with depth?

Water at the ocean surface is heated by the sun and warmed/cooled by contact with the atmosphere. That heat only slowly mixes downward because water is a poor conductor and because warm water is less dense than cold water — it floats on top. So a vertical slice of ocean typically looks like:

- **Mixed layer (surface to ~20–100 m):** wind and waves keep this layer well-stirred, so temperature is nearly uniform with depth.
- **Thermocline (below the mixed layer):** temperature drops sharply over a relatively thin band — this is the ocean's "temperature cliff."
- **Deep ocean (below ~1000 m):** cold (near 2–4°C) and nearly uniform, fed by polar water that sank long ago and slowly spreads along the bottom.

Analogy: think of a glass of iced coffee left still. The top warms in the sun, the bottom stays cold near the ice, and there's a thin zone in between where temperature changes fast. Stir it (wind) and the warm zone thickens; stop stirring and the sharp transition reappears.

2.2 Why do we care what's happening below the surface?

Not a list — a causal chain, worked through for the flagship example:

How does a temperature profile affect cyclone intensity? A cyclone draws its energy from evaporating warm surface water. As wind churns the sea, it pulls up water from just below the surface (upwelling/mixing). If the thermocline is deep and the subsurface water is nearly as warm as the surface (high **Tropical Cyclone Heat Potential**), the churned-up water is still warm, so the storm keeps getting fed and can intensify. If the thermocline is shallow, churning drags up cold water, the surface cools, and the storm's fuel supply is cut — this is one reason cyclones sometimes stall or weaken unexpectedly even over "warm" seas. **You cannot get this right from SST alone** — you need to know how warm it stays as you go down. This is the mechanistic reason INCOIS (which issues cyclone guidance for India) wants subsurface temperature, not just SST.

Similarly:

- **Marine heatwaves** are now defined and tracked at depth, not just at the surface, because subsurface heat has separate ecological and fisheries consequences (species displacement, coral stress at depth, changed fish migration).
- **Fisheries:** many commercially important species track the thermocline depth (their preferred temperature band), so thermocline depth maps directly inform fishing-ground forecasts (a related SIH-adjacent application area, e.g. Potential Fishing Zone advisories).
- **Ocean circulation / stratification:** the vertical density structure (driven by temperature and salinity) sets up the pressure gradients that drive currents; a bias in subsurface temperature propagates into circulation forecasts.
- **Data assimilation / forecasting:** operational ocean forecast systems (like GLORYS's production system) need frequent estimates of the full 3-D state to correct their models; a fast statistical proxy for the 3-D field is useful as a first guess or independent check.

2.3 What is actually wrong today? (the measurement gap)

System	What it measures	Coverage	Limitation
ARGO floats	Full T/S profile 0–2000 m	~4000 floats globally, drift with currents	One profile per float roughly every 10 days; large gaps, worse near coasts and in the northern Indian Ocean shipping lanes
Moored buoys (e.g. RAMA, OMNI/INCOIS)	Fixed-location T/S/currents time series, sometimes to several hundred m	Point locations only	Excellent temporal resolution, essentially zero spatial coverage
Gliders	Profile T/S along a controlled path	Regional, campaign-based	Expensive to operate, short deployments, sparse in time coverage across a basin
Ships (CTD/XBT)	Full-depth profiles, high accuracy	Along ship tracks only	Extremely sparse, expensive, biased toward shipping lanes
Satellites	Surface skin/near-surface fields only (SST from IR/microwave radiometers to ~10 μm–1 mm depth; SSH from altimeters; winds from scatterometers)	Near-global, daily	Cannot physically see below the surface — electromagnetic radiation used for remote sensing does not penetrate more than millimeters into seawater

Core physical answer to "why can't satellites just measure subsurface temperature?" Because heat and light from the ocean interior are almost completely absorbed or scattered within the top few millimeters to centimeters of water. A satellite radiometer literally cannot receive a signal that originated at 200 m depth. What satellites *can* see is the surface expression of processes whose causes and effects extend into the interior:

- **Thermocline displacement / mesoscale eddies:** a warm-core eddy bulges the sea surface upward (measurable by altimetry as +SSH) *and* pushes the thermocline deeper; a cold-core eddy does the opposite. So SSH is a statistical proxy for thermocline depth.
- **Vertical mixing:** strong winds (scatterometer-measured) mix the upper layers, which is visible as SST cooling and is also felt at depth.
- **Geostrophic transport:** SSH gradients drive currents (via geostrophic balance), and currents advect subsurface water masses — so surface currents carry information about what water is "downstream."
- **Air-sea coupling:** heat fluxes that set SST also set the depth and strength of the mixed layer.

None of these links are exact or one-to-one — that's exactly why this is a statistical/ML learning problem rather than a simple formula, and exactly why a **reanalysis product with real subsurface physics (GLORYS)** is needed to teach the model the actual (model-consistent) relationship.

3. The Problem in One Sentence, at Five Lengths

"We have continuous surface ocean satellite data, but we need continuous subsurface temperature profiles, and we currently cannot obtain the latter continuously because in-situ subsurface sensors are too sparse and satellites physically cannot see below the surface — so OceanEmbed learns the statistical link between the two from a reanalysis model, and checks itself against real ARGO floats."

- **10-second version:** We're teaching a computer to guess how warm the ocean is at different depths, just from what satellites see at the surface.
- **30-second version:** Satellites see the ocean's surface every day, but not what's underneath — and knowing what's underneath matters for cyclones, fisheries, and climate. Direct measurements below the surface (ARGO floats) are too sparse. We train a deep-learning model on a physics-based ocean reanalysis (GLORYS) to learn the relationship between surface patterns and subsurface temperature, then check the model against real independent float data.
- **1-minute version:** *(add the causal cyclone-heat-potential example from 2.2, plus the fact that this is INCOIS's operational problem.)*
- **5-minute version:** the full chain from 2.1–2.3 above.

4. What SIH Actually Wants Six Students to Build

INPUT → PROCESSING → EMBEDDING → MODEL → OUTPUT → VALIDATION

Stage	In	Happens	Out	Why necessary
Data acquisition	Raw NetCDF/Zarr from CMEMS, PODAAC, INCOIS, ERA5	Download, authenticate, cache	Raw files on disk	Nothing works without real data
Harmonization	Raw files, mixed grids/units/calendars	Coordinate renaming, unit conversion, calendar alignment	Consistent xarray Datasets	[A] explicitly required as "preprocessing and harmonization pipeline"
Regridding	Native-resolution fields	Bilinear/conservative interpolation to 0.25°×0.25°, daily	Common-grid fields	[A] explicit requirement
Missing-value handling	Gappy fields (clouds, swath gaps)	Masking, gap-fill or leave-as-NaN with mask channel	Clean tensors + validity mask	Satellite data always has gaps; ignoring this silently corrupts training
Normalization	Physical-unit fields	Per-channel standardization	Model-ready tensors	Stabilizes training
Embedding	Stacked surface channels	Encoder network (CNN/ViT/Autoencoder)	Latent vector/map	[A] explicitly named "satellite embedding engine"
Reconstruction	Latent representation	Decoder/regression head	15-depth temperature field	[A] the actual deliverable
Validation	Model output + ARGO	Compute skill metrics	RMSE/bias/correlation by depth	[A] explicit requirement, must be <i>independent</i> of training
Visualization	Predictions	Maps, profiles, depth-RMSE plots	Figures for demo	[A] "demonstration of a working PoC"
Serving/UI	Trained model	Lightweight API + frontend	Interactive demo	Needed to make the PoC "working" and presentable, not strictly named in PS text but implied by "software" category

5. Datasets — What They Actually Are

5.1 GLORYS (training target)

[D/A] GLORYS12 (the current flagship version, "GLORYS12V1") is produced by **Mercator Ocean International** for the EU's **Copernicus Marine Service (CMEMS)**. It is not a satellite product and not direct observation — it is the output of a physical ocean circulation model (**NEMO**), forced by atmospheric reanalysis (historically ERA-Interim, more recent versions use ERA5-class forcing), into which real observations are **assimilated**: along-track altimeter sea level anomaly, satellite SST, sea-ice concentration, and in-situ temperature/salinity profiles (including ARGO), corrected via a reduced-order Kalman filter plus a 3D-VAR large-scale bias correction. Native resolution is **1/12° (~8 km) with 50 vertical levels from the surface to ~5500 m**, covering 1993–present. GLORYS has been extensively validated and is considered one of the strongest global ocean reanalyses in comparative studies, though it still has known regional biases, especially where in-situ observation density (which corrects the model) is historically low — which includes parts of the North Indian Ocean before the ARGO era matured.

Ground truth vs. reanalysis target — do not conflate these. "Ground truth" means direct, calibrated physical measurement (an ARGO CTD sensor reading). GLORYS is a *model estimate constrained by observations* — it inherits real observations where they exist but **fills in the rest using ocean-model physics**, which has its own biases (mixing schemes, resolution limits, forcing errors). **If your model learns purely from GLORYS, in the strictest sense it learns "what GLORYS's model+data-assimilation system would predict," not "the true ocean."** This is exactly why the PS mandates independent ARGO validation — a model that matches GLORYS extremely well but matches independent ARGO poorly has learned GLORYS's biases, not ocean physics. Report both.

5.2 ARGO (independent validation)

[D/A] ARGO is an international array of ~4000 autonomous **profiling floats**. Each float drifts at a "parking depth" (usually ~1000 m), then every ~10 days descends to ~2000 m and rises to the surface, measuring temperature, salinity, and pressure continuously on the way up, then transmits the profile via satellite before sinking again. This gives high-accuracy, full-depth (to 2000 m) profiles, but at any given time and 0.25° grid cell, **the odds of having a float physically present are low** — which is precisely why ARGO alone cannot give you a continuous daily gridded field, but is exactly trustworthy enough to be the gold-standard check on a model.

INCOIS's Live Access Server (LAS) provides a **gridded ARGO** product for the Indian Ocean — i.e., ARGO's sparse profiles have already been objectively interpolated onto a regular grid by INCOIS. **[gap]** The exact resolution/depth levels/methodology of this specific INCOIS gridded ARGO product should be confirmed from the LAS documentation directly (search "INCOIS LAS gridded Argo") since it may differ

from GLORYS's depths and the PS's 15 standard levels. A general Indian-Ocean ARGO SST/gridded climatology paper found in our literature search reports **0.25°×0.25° monthly gridded ARGO products constructed using the DIVA (Data Interpolating Variational Analysis) interpolation method** — a similar approach is plausible for the LAS product but must be verified against the actual LAS metadata, not assumed.

Training vs validation, intuitively: imagine studying for an exam using a detailed answer key (GLORYS) that was itself written by a very good but imperfect teacher. Your "validation" should be a different, independently graded test (ARGO) — if you only check your answers against the same answer key you studied from, you'll always look perfect, even if you only memorized quirks of that particular teacher rather than the actual subject.

6. Surface Input Variables — Deep Dive

Variable	What it is	Physical meaning	Typical source	Why it may predict subsurface T	Native res. (typical)	Missing-data issue	Units to standardize to
SST	Sea surface temperature	Skin/near-surface heat state	OSTIA, CMEMS L4 SST (satellite-blended, gap-free)	Direct surface expression of the mixed layer's heat content; strongly correlated with upper ~50 m temperature	0.05°, daily	L4 (blended) products are already gap-filled; L3 swath products have cloud gaps	°C
SSS	Sea surface salinity	Surface freshwater/evaporation balance	SMOS/SMAP satellite retrievals or CMEMS multi-observation SSS	Salinity affects density/stratification, which modulates how deep surface heat mixes; monsoon freshwater input is a strong Bay-of-Bengal signal	0.125°–0.25°, often multi-day composites	Satellite salinity retrieval is noisy and coarser than SST; frequent gaps	PSU
SSH / SLA	Sea surface height / anomaly	Dynamic topography relative to geoid/mean	CMEMS DUACS (multi-altimeter merged)	Directly related to thermocline depth and eddy structure (warm eddy → high SSH → deep thermocline)	0.25°, daily (merged product)	Along-track native data is very sparse; must use the already-gridded DUACS L4 product	m
Surface currents (U,V)	Horizontal velocity at/near surface	Geostrophic + Ekman flow	Derived from SSH (geostrophic) + wind (Ekman), or CMEMS surface current product	Encodes transport of water masses; eddy rotation direction distinguishes warm-core vs cold-core structures that SSH alone can't fully disambiguate	0.25°, daily	Derived products inherit gaps from SSH/wind inputs	m/s
Surface winds (U,V)	10 m wind vector	Momentum/heat flux forcing	ERA5 (reanalysis) or ASCAT/scatterometer	Drives mixing and Ekman transport; sets mixed-layer depth	0.25°–0.5°, hourly-to-daily	ERA5 is essentially gap-free (it's a reanalysis, not raw satellite swath)	m/s

What the ML model should actually receive: stacked, co-registered channels of shape `[channels, lat, lon]` per day, each standardized (zero mean / unit variance per channel, computed on training data only), with an explicit **land/ocean mask** and a **data-validity mask** as additional channels so the network can learn to discount interpolated/gap-filled pixels rather than treating them as equally trustworthy as real observations.

7. Oceanography Fundamentals (compressed to what you need)

Concept	Intuition	Minimal math idea	Relevance to OceanEmbed
Density/equation of state	Colder & saltier water is denser and sinks	$\rho = \rho(T,S,p)$, nonlinear	Why T and S jointly control stratification — SSH matters even though target is T only
Stratification	Layering by density	Stable if $dp/dz < 0$ going down... (density increases downward)	Determines how easily surface signals mix down
Thermocline / mixed layer	The "shelf" of temperature vs depth	dT/dz sharply negative in thermocline	This is literally the shape your model must reproduce at each grid point
Geostrophic balance	Currents flow <i>along</i> , not down, pressure gradients (Coriolis effect)	$f v = (1/\rho) \partial p / \partial x$	Basis for deriving surface currents from SSH
Ekman transport	Wind-driven flow twists with depth due to Coriolis	Net transport 90° from wind (N or S hemisphere dependent)	Basis for wind→current→mixing pathway
Mesoscale eddies	Ocean "storms," 50–300 km, weeks-months lifetime	SSH anomaly $\pm 10\text{--}30$ cm	Primary physical mechanism linking SSH to thermocline depth
Upwelling/downwelling	Wind or coastal geometry pushes water up/down	Ekman pumping	Explains strong Arabian Sea seasonal (monsoon) subsurface signal
Heat content	Integrated T over depth	$\int \rho C_p T \, dz$	Related but not identical to the profile-reconstruction target
Arabian Sea / Bay of Bengal contrast	Arabian Sea: strong evaporation, upwelling during SW monsoon; Bay of Bengal: heavy river runoff, strong salinity stratification	—	These two sub-basins behave differently enough that model skill will likely differ regionally — a good validation/demo point

Do not turn this into a semester course — the table above plus the causal cyclone example in §2.2 is enough to defend the science to judges.

8. Remote Sensing Fundamentals (compressed)

[D] Passive sensors (infrared/microwave radiometers) measure naturally emitted radiation — used for SST. Active sensors (altimeters, scatterometers) transmit and measure the return signal — used for SSH (radar altimetry) and winds (scatterometry, via sea-surface roughness). Raw satellite data usually arrives as **swath (L1/L2)** data tied to the orbit track; operational users normally consume **gridded (L3/L4)** products where multiple satellites/passes have been merged and, for L4, gap-filled by interpolation — **you should use L4-equivalent, already-gridded products (CMEMS SST L4/OSTIA, DUACS SSH L4) rather than raw swaths**, both because the PS demands a uniform daily 0.25° grid and because building your own swath-to-grid pipeline is out of scope for a hackathon timeline. Clouds block infrared SST retrieval (not microwave), which is why blended L4 products exist — they use microwave + statistical interpolation to cover IR gaps.

9. Scientific Data Engineering

Formats: NetCDF (self-describing gridded scientific data, the de facto standard here), Zarr (chunked, cloud-friendly, good for large training sets), read via **xarray** (labelled multi-dimensional arrays; makes lat/lon/time-aware code readable) with **Dask** for out-of-core/parallel processing when data won't fit in RAM. **CDO (Climate Data Operators)** is a fast command-line tool for regridding/remapping before you even touch Python, often the pragmatic first step.

Regridding choices: bilinear (fast, smooths fields, fine for continuous fields like SST/SSH), conservative remapping (preserves area-integrated quantities, better when going from fine to coarse grids and when mass/heat conservation matters scientifically), nearest-neighbour (only for masks/categorical data, avoid for continuous physical fields).

Concrete answer to "six datasets, six grids, one ML-ready dataset":

- 1. Fix a common analysis grid: 0.25°×0.25°, lat 5–30°N, lon 45–105°E, daily timestamps (this *is* the target grid, since GLORYS/ARGO/PS already specify it).
- 2. For each source dataset: inspect native grid/CRS/units/calendar → regrid spatially (bilinear for SST/SSH/currents/winds; consider conservative if downsampling from very fine native SST) → resample temporally to daily (mean for sub-daily sources like ERA5 winds; nearest/linear-in-time for already-daily products) → convert units → rename coordinates to a shared convention ((lat), (lon), (time), (depth)).

- 3. Apply a shared land/ocean mask (derived from GLORYS bathymetry, since that's your target grid) to every input and output field so land pixels don't pollute loss computation.
- 4. Stack all variables into one aligned xarray Dataset on the common grid/time axis; write to Zarr chunked by time for efficient random-access training reads.
- 5. Only after this alignment step do you do train/val/test splitting and tensor generation (Part 10).

Dangers: regridding *before* unit conversion can silently corrupt values (e.g., interpolating raw Kelvin vs Celsius is fine, but interpolating across a bad fill-value like -32768 without masking first will destroy the field). Never interpolate over land into ocean cells or vice versa without a correct mask — this leaks unphysical values into your training data.

10. Machine-Learning Problem Formulation

Definitions: X = surface state on day *t*: tensor $\left([C, H, W]\right)$ where C = number of input channels (SST, SSS, SSH, U_curr, V_curr, U_wind, V_wind, + optional mask/lat-lon-as-channels), H×W = spatial grid over the domain. Y = subsurface temperature on day *t*: tensor $\left([D, H, W]\right)$ where D = 15 (the standard depths).

Model: $f_{\theta}(X) \rightarrow Y$

Formulation options, compared:

Formulation	Description	Pros	Cons	Fit for this PS
Pixel-wise regression	Independent regressor per grid cell (e.g., MLP or RF on local features)	Simple, fast, easy baseline	Ignores spatial context (can't "see" an eddy's shape)	Good baseline , not final model
Patch-based prediction	CNN over small tiles	Captures local spatial context, memory-efficient	Boundary artifacts between tiles	Reasonable if full-grid is too large for GPU memory
Full-grid prediction	CNN/U-Net/ViT ingests the whole domain at once, outputs the whole depth stack at once	Captures basin-scale structure (eddies, monsoon gyres) in one shot; matches how GLORYS/PS frame the problem	Needs enough GPU memory; domain is only ~100×240 cells at 0.25°, which is very tractable	[E] Recommended primary formulation
Profile-wise (per-location) sequence modelling	Treat depth as a sequence dimension, decode profile top-down	Can enforce smoothness/ordering along depth naturally (e.g. via an RNN/attention over depth)	More complex, marginal benefit given only 15 depths	Optional refinement, not required
Spatiotemporal (multi-day sequence in, sequence out)	Feed a window of days to exploit persistence/advection	Can improve skill, especially for slowly evolving eddies	Bigger engineering lift, more data needed, higher overfitting risk on short timelines	[E] Good "advanced" extension, not MVP

Recommendation: full-grid, single-day-in/single-day-out CNN or U-Net-style model, output shape $\left([15, H, W]\right)$, is the most defensible MVP formulation — it directly matches the PS's grid, is small enough to train on a laptop/Colab GPU, and lets you cleanly demonstrate the "embedding" stage as a distinct bottleneck layer.

11. What "Embedding" Actually Means Here

An **embedding** is a way of compressing a large, structured input into a smaller vector (or smaller feature map) that still preserves the information that matters for the task, while discarding irrelevant noise. Analogy: instead of describing a person's face pixel by pixel, you describe them by a handful of learned traits (face shape, eye spacing, etc.) — that's an embedding of the face.

In an **autoencoder**, an **encoder** network compresses input → **latent vector** (the embedding); a **decoder** reconstructs something from that latent vector. In **representation learning** more broadly, you don't need to reconstruct the original input — you can decode to a *different* target, which is exactly OceanEmbed's setup:



Why this helps: the raw 7-channel surface state is high-dimensional and noisy; the encoder is forced to learn a compact representation that only keeps information useful for reconstructing structure that correlates with subsurface temperature (eddy presence/sign, mixed-layer

depth signal, etc.), which acts as a form of regularization and also gives you an inspectable, visualizable intermediate product for your demo ("here is what the model learned to notice about the ocean surface before it even looks at depth").

12. Architecture Comparison

Architecture	Strengths	Weaknesses	Compute cost	Data need	Spatial structure	Long-range context	Interpretability	Fit
Plain CNN (encoder-decoder)	Simple, well-understood, strong local spatial priors	Limited long-range context without deep stacks	Low	Moderate	Good	Weak-moderate	Moderate	Strong MVP/recommended
U-Net (CNN + skip connections)	Preserves fine spatial detail via skips, standard for dense prediction	Slightly more complex than plain CNN	Low-moderate	Moderate	Excellent	Moderate	Moderate	Strong recommended (B)
Autoencoder (plain)	Directly matches the "embedding" narrative	Without skip connections, tends to blur fine structure	Low	Moderate	Moderate	Weak	High (clean latent)	Good for the embedding-focused framing
Vision Transformer (ViT)	Global receptive field from layer 1, good for basin-scale eddies	Needs more data / careful regularization at small dataset sizes; heavier compute	High	High	Weak locally without patches tuned right	Excellent	Low	Ambitious/advanced tier
Graph Neural Network	Naturally handles irregular ocean masks/coastlines	Less mature tooling, harder to implement well in hackathon time	High engineering cost	Moderate	Depends on graph construction	Good	Low-moderate	Not recommended for MVP timeline
CNN + Attention hybrid	Local CNN inductive bias + some global context	More hyperparameters to tune	Moderate	Moderate	Good	Good	Moderate	Good "ambitious" tier upgrade
ConvLSTM / temporal models	Exploits day-to-day persistence	Needs sequential training data, more complex pipeline	Moderate-high	High (long time series)	Good	Good (temporal)	Moderate	Advanced extension only
MLP baseline (per-pixel)	Trivial to implement	No spatial context at all	Very low	Low	None	None	High	Required baseline
Random Forest / XGBoost	Strong tabular baseline, robust, interpretable feature importances	No spatial context unless features engineered in	Low	Low-moderate	None (unless engineered)	None	High	Required baseline
Physics-informed (e.g. penalize static instability)	Improves physical plausibility	Extra complexity, may not clearly move core RMSE metric	Moderate	—	—	—	—	Optional, discussed in Part 15

Decision matrix logic [E]: Given a hackathon timeline, a domain of only ~100×240 grid cells, and the explicit PS mention of CNN/ViT/Autoencoder/GNN/attention as *options*, the defensible choice is: **U-Net-style encoder-decoder CNN as the primary architecture**, with the bottleneck explicitly exposed and visualized as "the embedding," plus a lightweight self-attention block inserted at

the bottleneck (cheap way to get some global context without full ViT cost/data requirements) as the "ambitious" version. This is Architecture B in Part 21.

13. Baselines (mandatory)

Before any deep model, establish:

1. **Climatology**: predict the long-term mean temperature-at-depth for that day-of-year and location (from GLORYS training years).
Answers "how much better than just knowing the season are we?"
2. **Persistence**: predict "same as N days ago" (less meaningful here since target is depth structure, but useful for later spatiotemporal extensions).
3. **Linear regression / Ridge**: per-depth linear regression from the 7 surface channels (globally or per grid cell).
4. **Random Forest / XGBoost**: nonlinear tabular baseline using surface channels (+ lat/lon/day-of-year) as features, trained per depth level.
5. **Simple MLP**: shared small MLP applied per-pixel.
6. **Simple CNN**: shallow 3-4 layer CNN, no fancy embedding stage.

Why mandatory: if your U-Net gets RMSE = 0.8°C at 100 m, that number is meaningless in isolation. If climatology alone already gets RMSE = 0.9°C at that depth (very plausible near the seasonal thermocline, which is fairly predictable), your "advanced" model is only marginally useful — and a judge asking "why deep learning?" deserves that comparison, not a vague appeal to sophistication.

14. Literature Review (materially relevant, verified via search — not exhaustive)

We prioritize papers that materially inform surface→subsurface reconstruction, not tangential ocean-AI papers.

Paper	Year	Venue	Problem	Inputs	Model	Depth range	Relevance	Adopt	Avoid
"Reconstructing High-Resolution Ocean Subsurface and Interior Temperature and Salinity Anomalies From Satellite Observations"	Not confirmed from accessible snippet	IEEE Xplore (IEEE TGRS-class)	DNN retrieval of subsurface T/S anomaly, Pacific, 1/4° and 1/12°	Multi-source satellite surface fields (SSH, SST, SSS, wind)	Deep neural network	Near-surface to 4000 m	Very high — near-identical problem framing to OceanEmbed, at larger basin scale	Multi-depth DNN with satellite-only inputs is viable at high resolution	Details of exact architecture/loss "not available from the accessible source"
"Subsurface Temperature Reconstruction for the Global Ocean from 1993-2020 Using Satellite Observations and Deep Learning" (DORS product)	2022	Remote Sensing (MDPI)	Global 3-D thermal structure reconstruction	Multisource remote sensing + Argo gridded data	ConvLSTM	Upper 2000 m	High — explicit use of ConvLSTM and Argo, long time series (1993-2020)	Temporal (ConvLSTM) modelling is a proven extension path if MVP succeeds	Full 2000 m + 28-year training is beyond hackathon scope; scale down
"Reconstruction of ocean temperature and salinity profiles in the Northern South China Sea using satellite observations"	2022	Frontiers in Marine Science	Regional T/S profile reconstruction	SST + SSH (CMEMS), historical Argo (WOD18) for climatology base	Linear regression on top of a static climatology	Full profile	Medium-high — good regional baseline design pattern (climatology + linear correction), thermocline error ~1.8°C noted as their achieved standard-deviation ceiling	The "climatology + statistical correction" design is a strong, simple baseline worth replicating	Their skill is bounded by linear regression — expect it to underperform a CNN, use as a baseline reference number, not a target to imitate architecturally
"Satellite-based reconstruction of high-resolution ocean subsurface temperature using spatiotemporal graph attention networks"	2026	ScienceDirect (Journal not confirmed from snippet)	High-res subsurface T reconstruction	Not fully confirmed from accessible snippet	Spatiotemporal Graph Attention Network	Not confirmed	Medium — shows GNN/attention approaches are an active current research direction (2026), supports listing GNN as a legitimate "ambitious" option rather than pure hype	GAT-style local graph attention is a plausible extension beyond U-Net if time allows	Full architecture/results details "not available from the accessible source" — do not cite specific numbers from this paper without reading the full text
"Seeing through the Sea with Satellites:	2024	ESSDD (Copernicus preprint)	Physics-informed satellite-only	Satellite surface observations	Physics-informed algorithm (not	Upper ocean	Medium — directly relevant to	Consider a simple physical	Full physics-informed algorithm re-

Paper	Year	Venue	Problem	Inputs	Model	Depth range	Relevance	Adopt	Avoid
Reconstructing Ocean Subsurface Temperature and Salinity with Satellite Observations"			reconstruction of upper-ocean T/S		a pure black-box net)		whether physics-informed constraints are worth the complexity (Part 18)	sanity check (monotonic-with-depth cooling below the mixed layer) inspired by this line of work, without committing to a full physics-informed loss	implementation is likely out of scope for a hackathon
GLORYS12 description paper (Jean-Michel et al., "The Copernicus Global 1/12° Oceanic and Sea Ice GLORYS12 Reanalysis")	2021	Frontiers in Earth Science	Describes GLORYS12 itself	—	NEMO ocean model + reduced-order Kalman filter + 3D-VAR bias correction	Full depth, 50 levels	High — this is your training-target dataset's own reference paper, cite it for methodology when explaining GLORYS to judges	Cite directly when explaining what GLORYS is and its assimilation scheme	—

Honest gap note: several strong candidate papers (e.g., the IEEE TGRS DNN paper, the 2026 graph-attention paper) had only abstract-level snippets accessible through search — **do not present numeric RMSE/skill results from these papers to judges without pulling and reading the actual paper**, since exact figures could not be verified here.

15. Literature Synthesis

What's already solved: the basic feasibility question — "can surface satellite fields predict subsurface temperature with useful skill using deep learning" — is well-established in the literature (multiple independent groups, multiple ocean basins, multiple architectures from linear regression through ConvLSTM to graph attention networks, spanning 2022–2026). **This is not novel research territory in the abstract.**

What remains genuinely open / regionally specific: a **North Indian Ocean-specific, 0.25°/daily, 15-standard-depth, GLORYS-trained-ARGO-validated** implementation targeted specifically at INCOIS's operational depth levels and domain does not appear to be a publicly available, off-the-shelf product — most published work targets the Pacific, South China Sea, or global scale at different depth/resolution choices. That regional/operational specificity is your realistic contribution (see Part 18/Novelty).

Consistent findings across papers (general pattern, not all individually re-verified here): skill (correlation, RMSE) is typically **highest near the surface and mixed layer, degrades through the thermocline (roughly 100–300 m, where vertical gradients are steep and small depth errors translate into large temperature errors), and often improves again below ~500–700 m where the ocean is more uniform and slowly varying.** This U-shaped (or "worst-in-the-middle") error-vs-depth pattern is a very defensible, literature-consistent expectation to state to judges rather than a specific number.

Role of Argo vs reanalysis, restated: reanalysis (GLORYS) supplies dense, gap-free training targets everywhere; ARGO supplies the sparser but independently-trustworthy check. Multiple papers explicitly combine both, exactly as this PS mandates.

Realistic research gap for a six-person student team: not a new architecture, but a well-executed, honestly-validated, INCOIS-operational-parameter-matched regional implementation with a clean demo — and, if time allows, a genuine (if modest) empirical question such as "does adding surface currents (U,V) meaningfully improve skill over SST+SSH+SSS alone in the Bay of Bengal specifically?" — this is answerable in a hackathon timeframe via an ablation study and is a legitimate, evidence-backed claim of contribution.

16. Train / Validation / Test Strategy

Split type	Description	Risk
Random (i.i.d.) split across all grid-cell-days	Simple, but...	Severe leakage: adjacent grid cells and adjacent days are highly spatially/temporally autocorrelated (an eddy persists for weeks and spans many grid cells) — random splitting lets the model "cheat" by seeing near-duplicate examples in training and test
Temporal split	Train on years 1993–20XX, test on later years (or hold out full seasons)	Best single answer to "does the model generalize to unseen ocean conditions," directly testable, standard in the literature
Spatial split	Hold out whole sub-regions (e.g., train Arabian Sea, test Bay of Bengal, or checkerboard blocks)	Tests spatial generalization; harder, but very convincing to judges if it still works reasonably
Spatio-temporal split	Combine both: hold out later years AND certain regions	Most rigorous, [E] recommended primary strategy: hold out the most recent 1–2 full years entirely for test, and within the training years use blocked spatial cross-validation blocks (not random single pixels) for model selection

Explicit leakage prevention: compute normalization statistics only on training years; never let a validation-year statistic leak into preprocessing done "globally"; when doing any regridding/interpolation, apply it before splitting only if it's purely spatial-structural (doesn't use future information) — temporal gap-filling interpolation must not use future days to fill past days across a split boundary.

17. Loss Functions

- **MSE:** standard, penalizes large errors heavily — reasonable default.
- **MAE:** more robust to outliers (e.g., rare extreme ARGO or GLORYS values); good complement metric even if not the training loss.
- **Huber:** middle ground, good practical default for noisy geophysical targets.
- **Depth-weighted loss:** should 5 m and 1000 m be weighted equally? [E] No, and here's the reasoning, not an assumption: the standard depths are not evenly spaced (5 m steps near the surface, 200–300 m steps below 500 m) and the *physical* variability of temperature is far larger and more dynamically important near the surface/thermocline than in the deep, nearly-isothermal ocean. Two defensible weighting choices: (a) inverse-variance weighting (weight each depth by 1/GLORYS-climatological-variance at that depth, so the loss doesn't get dominated by the naturally larger swings near the surface), or (b) uniform weighting but **always report depth-resolved metrics separately** so no single number hides thermocline-region failure. [E] **Recommendation: train with uniform or mildly variance-weighted loss, but never report a single averaged RMSE as the headline metric — always show the depth-wise breakdown.**

18. Evaluation Plan

Report, at minimum: RMSE, MAE, Bias, Pearson correlation, and (where sample size allows) R^2 , each broken down by **depth, season, and sub-region (Arabian Sea vs Bay of Bengal)** — never a single overall number as the headline result.

Proposed plots: predicted-vs-observed profile at sample ARGO locations; depth-vs-RMSE curve; depth-vs-bias curve; depth-vs-correlation curve; spatial error maps per depth; predicted temperature maps vs GLORYS/ARGO maps side by side; vertical cross-sections along a chosen latitude/longitude line. Taylor diagrams are a nice-to-have if time allows (they compactly show correlation + variance ratio + RMSE together, and judges familiar with atmospheric/ocean science recognize them immediately, which helps credibility).

19. Physical Sanity Checks

[E] **Recommendation, with cost/benefit stated rather than assumed:** don't build a full physics-informed loss function (high complexity, uncertain payoff in the timeline) — instead add a **cheap, high-value post-hoc physical plausibility check:** verify the predicted profile is monotonically non-increasing with depth below the mixed layer for the vast majority of grid points/days (temperature inversions at depth are physically rare in this domain outside specific known phenomena), and flag/report the fraction of predictions that violate this. This costs almost nothing to implement, gives you a genuinely impressive judge-facing plot ("<X% of our predictions show unphysical inversions"), and demonstrates scientific rigor without committing engineering time to a custom physics-informed loss.

20. Uncertainty

[E] **Recommendation:** full probabilistic modelling (quantile regression, MC-dropout ensembles, CRPS scoring) is valuable but likely too costly for the base timeline. Propose it explicitly as the "advanced" tier: a simple, cheap proxy is **MC-Dropout at inference time** (keep dropout active, run N forward passes, report the spread as an uncertainty estimate) — this requires **zero architecture change**, only inference-time behavior, making it a very good "if time permits" addition rather than an MVP requirement.

21. Final Architecture Recommendation

Architecture A — simplest viable (fallback / guaranteed to finish): shallow CNN encoder-decoder, no fancy embedding visualization, MSE loss, single depth-averaged skill report. Use this if the team is badly behind schedule.

Architecture B — recommended: U-Net-style encoder-decoder with an explicit bottleneck latent layer (this is your "embedding"), Huber loss with depth-aware weighting, full depth/season/region-resolved evaluation, monotonicity sanity check, MC-Dropout uncertainty as a bonus if time allows. **[E] This is our top recommendation** — it satisfies every explicit PS requirement, is buildable and trainable within a hackathon compute budget (laptop GPU or free-tier Colab), and gives you a clean, visualizable embedding to show judges.

Architecture C — ambitious: Architecture B's U-Net backbone with a lightweight self-attention block at the bottleneck (cheap global-context boost, avoids full ViT data-hunger), plus a ConvLSTM temporal extension trained on a short sequence window if the team has bandwidth in the final days, plus the ablation study (does adding currents help?) as the "genuine contribution" claim.

Why not go straight to full ViT or GNN? Both explicitly appear as PS-suggested options, so mentioning them is legitimate — but both are data-hungry or engineering-heavy relative to the domain size (~24,000 grid cells) and hackathon timeline; a U-Net gets you comparable or better practical skill at a fraction of the implementation risk, which is the correct engineering trade-off to defend to judges.

22. System Architecture, Stack, and Compute

Layers: Data layer (raw + harmonized Zarr store) → Processing layer (regridding/normalization scripts) → ML layer (PyTorch training/eval code) → Model serving layer (FastAPI wrapping the trained model) → Frontend (map + profile viewer) — keep the **research/ML pipeline** (notebooks, training scripts, evaluation) architecturally separate from the **demo/application layer** (API + frontend) so judges can inspect either independently, and so a frontend bug never blocks the ML story.

Stack [E], each justified:

- **Python** — ecosystem requirement, no real alternative for this domain.
 - **PyTorch** over TensorFlow — better ecosystem fit for quick iteration and easier custom loss/masking logic for a small team; either is defensible, PyTorch is our pick for familiarity/speed of hackathon iteration.
 - **xarray + NumPy + Dask** — necessary for correct, labelled multi-dimensional geoscience data handling; Dask only if a laptop can't hold the harmonized dataset in memory (if it fits, skip Dask to save complexity/time).
 - **CDO or xESMF** for regridding — faster/more reliable than hand-rolled interpolation.
 - **NetCDF4/Zarr** — required scientific data formats.
 - **Cartopy + Matplotlib** for scientific maps; **Plotly** only if you want interactive frontend charts — don't build both, pick one per surface.
 - **FastAPI** for a minimal inference API — lightweight, fast to stand up.
 - **A simple static or lightweight React frontend** — only as complex as needed to show a map + a click-to-see-profile interaction; do not over-invest here relative to the ML story.
 - **Docker** — optional but good practice for reproducibility; skip if it threatens the timeline.
 - **GitHub** — required for team collaboration and judge-visible history.
 - **Compute:** domain is small (~100×240 cells, 15 depths, 7 input channels) — a free-tier Colab/Kaggle GPU (T4-class, ~16 GB VRAM) is more than sufficient for Architecture A/B; storage for several years of harmonized daily 0.25° data over this domain is on the order of a few GB to low tens of GB depending on year range chosen, not TBs — **[E] estimate, not benchmarked; verify against your actual downloaded data size early.**
-

23. GitHub / Repository Setup, From Zero

1. Create a GitHub organization or repo owned by one member, add all six as collaborators.
2. Repo name: something short and matching the PS spirit, e.g. `oceanembed-sih26066`.
3. Initialize with README, `.gitignore` (Python template + data/ + *.nc + *.zarr exclusions), and a permissive LICENSE (MIT) unless your institution requires otherwise.
4. Protect `main`; require PRs to merge.
5. Create a `dev` branch; feature branches off `dev` (`feat/regridding`, `feat/unet`, etc.).
6. Create GitHub Issues for every atomic task in Part 24; group into Milestones (Data pipeline, Baselines, Model v1, Evaluation, Demo).
7. Use a GitHub Project board (Kanban: To do / In progress / Review / Done).
8. Locally: `git clone`, create a Python virtual environment (`python -m venv .venv` or `conda create`), install dependencies, `pip freeze > requirements.txt`.
9. Establish a commit convention (e.g. Conventional Commits: `feat:`, `fix:`, `data:`, `docs:`) and a lightweight PR review rule (at least one other teammate approves before merge to `dev`).

Repository structure [E, adapt as needed]:

```

oceanembed/
├── README.md
├── LICENSE
├── .gitignore
├── requirements.txt
├── configs/          # YAML configs for data domain, depths, model hyperparams
├── data/             # gitignored; raw/ and processed/ subfolders, or pointers/download scripts only
├── notebooks/        # exploration only, not the source of truth
├── src/
│   ├── data/         # download + harmonization + regridding
│   ├── preprocessing/ # normalization, masking, tensor generation
│   ├── models/        # architecture definitions (baseline, unet, etc.)
│   ├── training/      # training loop, loss functions
│   ├── evaluation/    # metrics, plots, sanity checks
│   └── visualization/ # maps, profile plots
├── scripts/          # CLI entrypoints (train.py, evaluate.py, etc.)
├── tests/             # unit tests for regridding, masking, metrics
├── api/              # FastAPI serving layer
├── frontend/         # demo UI
├── docs/             # this document + judge Q&A + architecture diagrams
└── models/           # gitignored trained weights, or tracked via releases/LFS

```

24. Implementation Roadmap (atomic tasks)

Day 0 (setup):

1. Log into sih.gov.in and open the actual SIH26066 attached dataset table — resolve the Part 1 gap.
2. Create GitHub repo, board, and branch structure per Part 23.
3. Register/verify access to CMEMS (Copernicus Marine Service) account for GLORYS + SSH/SSS products.
4. Locate and test access to INCOIS LAS gridded ARGO endpoint.
5. Locate ERA5 access (via Copernicus Climate Data Store) for winds if not using a satellite wind product directly.
6. Fix the exact domain/time range for the MVP (e.g., 2015–2022 for training, 2023 held out for test) and write it into `configs/domain.yaml`.

Data pipeline: 7. Download a small pilot subset (e.g., 1 month) of each dataset. 8. Write a loader per dataset; print dimensions/coords/units to confirm understanding. 9. Standardize coordinate names (`lat`, `lon`, `time`, `depth`) across all loaders. 10. Convert units to the shared convention (Part 6 table). 11. Build the land/ocean mask from GLORYS bathymetry on the 0.25° domain grid. 12. Implement regridding to 0.25°×0.25° for every non-native-resolution input. 13. Implement daily temporal resampling for every non-daily input. 14. Crop every dataset to the exact domain box (5–30°N, 45–105°E). 15. Write validity-mask channels capturing where data was gap-filled vs. observed. 16. Save the harmonized multi-year dataset to Zarr; write and run a unit test that checks shapes/coords/no unexpected NaNs on land-excluded cells. 17. Implement per-channel normalization (fit on training years only). 18. Implement train/val/test splitting per Part 16 strategy; save split indices to disk (reproducibility). 19. Build the PyTorch Dataset/DataLoader that yields `(X: [7,H,W], Y: [15,H,W], mask)` tensors. 20. Run and log all six baselines from Part 13 on the val split; record numbers before touching any deep architecture.

(Continue with model-building, training, evaluation, API, frontend, demo-script tasks — each following the same atomic-action pattern; not exhaustively enumerated here for length, but every subsequent task should be written in this same "download X → inspect → transform → save → test" granularity before being assigned to a team member.)

Critical path: dataset access → harmonization → regridding/alignment → baseline → Architecture B model → evaluation framework → demo.

Parallelizable: frontend/API scaffolding, literature review writeup, and judge-Q&A prep can all proceed in parallel with the data pipeline once the target schema (Part on data schema) is fixed early.

25. Team of Six — Role Allocation

Role	Responsibilities	Key deliverable	Depends on
Data engineering lead	Download, harmonize, regrid all datasets; own the Zarr store and schema	Clean, versioned harmonized dataset	Dataset access confirmed (Day 0)
Oceanography/domain researcher	Own Parts 2-5/7-8 of this doc for the team; validate physical plausibility checks; write judge-facing science explanations	Science narrative + sanity-check module	Data pipeline outputs to sanity-check against
ML/model developer	Implement baselines + Architecture B/C, training loop, loss functions	Trained models + checkpoints	DataLoader from data engineering
Experimentation/evaluation lead	Own the evaluation framework, ARGO validation, plots, ablations	Metrics tables + all evaluation plots	Trained models
Backend/integration engineer	FastAPI serving layer, model packaging, connects frontend to model	Working inference API	Trained model checkpoint
Frontend/visualization engineer	Map + profile UI, demo flow, presentation polish	Working demo UI	API endpoints

Keep the ML developer and evaluation lead in constant sync (weekly/daily standups) since baseline numbers gate whether Architecture B is worth pursuing further; avoid letting the frontend/backend pair become idle early by having them build against a mocked API contract before the real model is ready.

26. MVP vs. Strong vs. Advanced

- **MVP (must exist):** harmonized dataset, all six baselines, Architecture A or B trained and evaluated with depth-resolved metrics against independent ARGO, one region/one season demo, monotonicity sanity check, basic map + profile plots (even static images are acceptable).
- **Strong prototype:** Architecture B fully realized (visualized embedding bottleneck), spatio-temporal train/test split, interactive frontend (click map → see profile), ablation study (which input variables matter most).
- **Advanced (only if MVP+strong are solid early):** self-attention/ConvLSTM extension, MC-Dropout uncertainty, Taylor diagrams, multi-region comparison (Arabian Sea vs Bay of Bengal skill difference as a headline finding).
- **Explicitly do NOT build:** a full production Kubernetes deployment, a custom database when files suffice, a mobile app, a fully physics-informed differentiable-ocean-model loss, or a from-scratch GNN pipeline — all high effort, low marginal credibility gain relative to nailing the core pipeline and honest validation.

27. Demo Design

Sequence: (1) show the raw surface satellite maps for a chosen day, (2) show them flowing into the harmonization pipeline, (3) visualize the learned embedding/bottleneck as a small multi-channel feature map ("this is what the model distilled from the surface"), (4) show the predicted 3-D temperature field / vertical cross-section, (5) overlay real ARGO float locations and show predicted-vs-observed profile at those points side by side, (6) show the depth-vs-RMSE/correlation plot as the headline validation evidence, (7) show the spatial error map, (8) close with one concrete real-world hook (cyclone heat-potential or fisheries) tying the numbers back to why INCOIS needs this.

28. Judge Cross-Questioning (representative set — expand with your own team's rehearsal)

(Short answer / deep answer / what not to say, for each.)

1. **Why deep learning instead of a simpler statistical method?** *Short:* Because the surface-to-subsurface relationship is nonlinear and spatially structured; our baselines (linear regression, RF) quantify exactly how much DL gains us. *Deep:* Linear methods can't capture eddy-dependent, spatially-varying relationships; our ablation shows DL beats RF by X (report your actual number). *Don't say:* "Because deep learning is state of the art" with no baseline comparison — this is exactly the trap Part 15 exists to avoid.
2. **Why GLORYS as the training target and not something else?** *Short:* It's the officially recommended, DOI-cited dataset, and it's the best-validated eddy-resolving global reanalysis available with full depth coverage. *Deep:* explain assimilation scheme (Part 5.1) and cite its known regional strengths/limitations honestly. *Don't say:* "GLORYS is ground truth" — it is a model-constrained reanalysis, not direct observation.
3. **Why ARGO for validation and not just a GLORYS holdout?** *Short:* To prove we learned real ocean physics, not GLORYS's internal biases. *Deep:* explain the "answer key vs. independent exam" analogy (Part 5.2). *Don't say:* claim validation is "perfect" — ARGO itself has sparse coverage and its own representativeness error at a 0.25° grid cell.
4. **Why not a random train/test split?** *Short:* Spatial/temporal autocorrelation would leak information and inflate our reported skill. *Deep:* explain spatio-temporal blocked splitting (Part 16).

5. **What happens below 500 m — does skill collapse?** *Short:* Expect improved, not worse, skill deep down, because deep water is more uniform; the hardest zone is the thermocline (~100–300 m). *Deep:* cite the literature-consistent U-shaped error-vs-depth pattern (Part 15), show your actual depth-vs-RMSE plot.
6. **What is your genuine novelty?** *Short:* Not a new architecture — a regional, INCOIS-parameter-matched, honestly-validated operational pipeline plus an ablation study on which surface variables matter most in this basin. *Don't say:* claim to have invented a fundamentally new method — the literature review shows this is an active, multi-group research area already.

(Continue rehearsing similarly for: scalability, security/data integrity of the pipeline, why not existing systems like existing gridded-Argo products alone, deployment plan, handling of missing satellite data operationally, extreme-event/monsoon generalization, and computational cost at operational (basin-wide, daily) scale.)

29. Failure Modes

Failure	Cause	Detection	Mitigation
Data leakage inflates reported skill	Random split despite spatial/temporal autocorrelation	Suspiciously high val skill vs. genuinely independent ARGO skill	Enforce spatio-temporal blocked splitting (Part 16); always report ARGO-independent numbers as headline
Model just learns GLORYS bias	Sole reliance on GLORYS-internal validation	Good GLORYS-val score but poor ARGO score	ARGO validation mandatory and reported first
Missing satellite data silently corrupted training	No validity mask, fill-values interpolated as real numbers	Physically implausible extreme values in inputs	Explicit validity-mask channel + fill-value auditing before regridding
Poor deep-ocean or thermocline performance	Inherent difficulty of thermocline dynamics, insufficient depth-aware loss weighting	Depth-vs-RMSE plot shows spike	Depth-weighted loss (Part 17), report honestly rather than hide behind average RMSE
Insufficient GPU/compute near deadline	Underestimated dataset size or model complexity	Training doesn't finish in time	Domain is small — fall back to Architecture A, use Colab/Kaggle free GPU tiers, keep a "demo mode" with a reduced spatial/temporal slice
Weak Bay of Bengal skill due to strong salinity stratification effects not well captured	SSS input quality/resolution limitations	Regional error map shows Bay of Bengal worse than Arabian Sea	Report it honestly as a finding, not a bug to hide; discuss in "where we could be wrong"
Distribution shift to extreme/monsoon conditions	Training years may underrepresent extreme monsoon or heatwave conditions	Seasonal skill breakdown shows monsoon months worse	Explicitly test and report seasonal skill; note as a known limitation

30. What We Can Claim

Can confidently claim (after building): "our pipeline correctly harmonizes N heterogeneous datasets to a common 0.25°/daily grid," "our baselines establish a defensible skill floor," "our deep model beats [specific baseline] by [measured amount] at [specific depths/regions] on independent ARGO validation," "predictions are physically plausible in >X% of cases (monotonicity check)."

Requires experimental evidence before claiming: "our model generalizes well to unseen years/regions" (only if your actual spatio-temporal holdout supports it), "surface currents meaningfully improve skill" (only after running the ablation).

Must NOT claim without evidence: "this is more accurate than existing INCOIS operational products" (you have no access to compare against those), "this works at true real-time operational latency" (a hackathon PoC is not a production system), "this generalizes beyond the North Indian Ocean domain trained on," "GLORYS is ground truth" (Part 5.1).

31. Novelty — Honest Assessment

[Brutally honest per the brief's own instruction]: the *general* capability — reconstructing subsurface ocean temperature from surface satellite fields via deep learning, trained on reanalysis, validated on ARGO — is **not novel**; it is an active multi-group research area with several published implementations across 2022–2026 spanning the Pacific, South China Sea, and global scale, using architectures from linear regression through ConvLSTM to graph attention networks. A student team should not claim to have invented this approach.

What is realistically, defensibly novel/contributory for this project:

- A North Indian Ocean-specific implementation matched exactly to INCOIS's 15 operational standard depths and 0.25°/daily operational grid, which does not appear as an off-the-shelf public product.

- An explicit, visualized "satellite embedding" bottleneck as a first-class, inspectable artifact (many published papers treat the network as an end-to-end black box rather than foregrounding the embedding, despite the PS explicitly asking for one).
- A genuine, executable ablation study (which surface variables matter most, Arabian Sea vs. Bay of Bengal) as real, small-scale original evidence.
- A rigorously honest spatio-temporal validation methodology and physical-plausibility sanity checking, presented transparently rather than glossed over — itself a legitimate engineering contribution in a field where leakage and over-claiming are common failure modes (see Part 29).

32. Where Our Reasoning Could Be Wrong

- **GLORYS itself may have systematic North Indian Ocean biases** (as noted, in-situ observation density historically constrains assimilation quality, and this region has complex monsoon-driven dynamics) — if GLORYS is biased in a particular season/region, the model will learn and reproduce that bias, and ARGO validation, while a genuine check, is itself sparse enough in this basin that a bias could partly evade detection. *Test:* explicitly stratify ARGO validation by season and sub-region and look for systematic sign-consistent bias, not just aggregate RMSE.
- **0.25°/daily may not be fine enough to resolve the mesoscale eddies that are mechanistically central to the surface-subsurface link** (eddies can be tens of km across, near the grid's resolution limit) — if so, the "embedding" may be learning a coarser, climatology-dominated relationship rather than true eddy-resolved dynamics. *Test:* check whether skill is meaningfully better than climatology specifically in high-eddy-activity regions/seasons vs. low-activity ones.
- **The 15 fixed standard depths, chosen for their oceanographic/operational relevance, are unevenly spaced, which could bias a naively-averaged loss toward the surface-dense depths** — addressed via depth-aware weighting and always reporting per-depth metrics (Part 17), but the specific weighting scheme chosen is itself a modeling assumption, not a proven-optimal choice; different weightings should be quickly compared as a sensitivity check if time allows.
- **A U-Net's local receptive field may under-capture basin-scale monsoon-driven signals** that a full ViT or GNN might capture better — the choice of Architecture B over C/ViT is a pragmatic timeline trade-off, not a claim that U-Net is scientifically superior; this should be stated plainly to judges rather than defended as if it were obviously correct.

33. Learning Roadmap for the Team

1. Python scientific computing basics (NumPy, Pandas) — foundation.
2. xarray + NetCDF — you cannot touch the actual data without this.
3. Basic physical oceanography (Part 7 table) — enough to speak credibly, not a full course.
4. Remote sensing basics (Part 8) — enough to justify dataset choices.
5. Geospatial regridding concepts — needed before writing the pipeline.
6. CNNs and encoder-decoder architectures — core model family.
7. Autoencoders / embeddings conceptually — needed to frame "the embedding" correctly.
8. (Optional, advanced tier only) Transformers/ViT, ConvLSTM, GNN basics.
9. Model evaluation methodology for geoscience (why not just accuracy/R² blindly; depth/season/region-resolved reporting) — critical, often skipped by hackathon teams, a strong differentiator.
10. Scientific visualization (Cartopy/Matplotlib basics for maps).
11. Minimal API/deployment (FastAPI) — enough to serve a model, not a full backend course.

What can be skipped: deep numerical-ocean-modelling theory (NEMO internals), full CRS/geodesy theory beyond "know your lat/lon grid and don't mix projections," building your own satellite retrieval algorithms (use existing L4 products).

34. Start Here Tomorrow Morning

1. Log into sih.gov.in with your team's registered credentials and open the SIH26066 page's attached dataset section — write down the actual official dataset list/links (resolves the Part 1 gap).
2. Open and read the GLORYS DOI page (<https://doi.org/10.48670/moi-00021>) and register a Copernicus Marine Service account if you don't have one.
3. Find and bookmark the INCOIS Live Access Server (LAS) gridded ARGO product page; confirm its resolution and depth levels.
4. Create the GitHub repository with the structure in Part 23.
5. Set up the GitHub Project board with columns and the Day-0/data-pipeline issues from Part 24 (items 1–20) entered as individual issues.
6. Assign the six roles from Part 25.
7. Each member installs Python (3.10+), creates the shared virtual environment, and confirms `import xarray, numpy, torch` works.
8. Data engineering lead: download one month of GLORYS temperature over the domain box as a pilot file.
9. Data engineering lead: download one month of CMEMS SSH (DUACS) over the same domain/time.
10. Data engineering lead: download one month of an SST L4 product (e.g., OSTIA) over the same domain/time.

11. Data engineering lead: download one month of SSS (satellite or CMEMS multi-obs product) over the same domain/time.
12. Data engineering lead: download or access one month of ERA5 10 m winds over the domain/time.
13. Data engineering lead: locate/derive surface currents for the same period.
14. Print `(.dims)`, `(.coords)`, and units for every one of the five pilot files — confirm you actually understand what you downloaded before writing any pipeline code.
15. Oceanography researcher: write a one-page plain-language explanation of the problem (Parts 2–4 condensed) for use in the pitch, and start collecting the 3–4 papers from Part 14 to actually read in full (not just abstracts).
16. ML developer: implement the climatology and linear-regression baselines on the one-month pilot data (even with a tiny sample) to validate the end-to-end pipeline shape before scaling up.
17. Evaluation lead: implement the RMSE/MAE/correlation/bias metric functions and the depth-vs-metric plotting function, tested on dummy arrays.
18. Backend engineer: scaffold a FastAPI app with one placeholder `(/predict)` endpoint returning dummy data, so the frontend can start integrating immediately.
19. Frontend engineer: scaffold the map + profile-click UI against the dummy endpoint.
20. End of day: whole-team 15-minute sync — confirm the actual official dataset list found in step 1 matches (or doesn't match) the Part 6/14 assumptions in this document, and adjust the data-pipeline plan accordingly before writing more code.

References (sources actually consulted for this document)

- SIH26066 official problem statement (mirrored) — <https://sih2026.vuce.in/ps/SIH26066> (source of truth: <https://sih.gov.in/sih2026PS>)
- GLORYS12 dataset DOI — <https://doi.org/10.48670/moi-00021>
- Jean-Michel et al., "The Copernicus Global 1/12° Oceanic and Sea Ice GLORYS12 Reanalysis," *Frontiers in Earth Science*, 2021 — <https://www.frontiersin.org/journals/earth-science/articles/10.3389/feart.2021.698876/full>
- "Reconstructing High-Resolution Ocean Subsurface and Interior Temperature and Salinity Anomalies From Satellite Observations," *IEEE Xplore* — <https://ieeexplore.ieee.org/document/9542964/>
- "Subsurface Temperature Reconstruction for the Global Ocean from 1993 to 2020 Using Satellite Observations and Deep Learning," *Remote Sensing (MDPI)*, 2022 — <https://www.mdpi.com/2072-4292/14/13/3198>
- "Reconstruction of ocean temperature and salinity profiles in the Northern South China Sea using satellite observations," *Frontiers in Marine Science*, 2022 — <https://www.frontiersin.org/journals/marine-science/articles/10.3389/fmars.2022.945835/full>
- "Satellite-based reconstruction of high-resolution ocean subsurface temperature using spatiotemporal graph attention networks," *ScienceDirect*, 2026 — <https://www.sciencedirect.com/science/article/pii/S1569843226001536>
- "Seeing through the Sea with Satellites: Reconstructing Ocean Subsurface Temperature and Salinity with Satellite Observations," *ESSDD preprint*, 2024 — <https://essd.copernicus.org/preprints/essd-2024-334/>
- "Generation and Assessment of ARGO Sea Surface Temperature Climatology for the Indian Ocean Region," *Oceanologia*, 2023 — <https://doaj.org/article/dd4932bb230e4982a163f33da2d561d5>
- Ocean Temperature overview, NASA PO.DAAC/JPL — <https://podaac.jpl.nasa.gov/SeaSurfaceTemperature>

Not available from the accessible source / needs first-hand verification by the team: exact resolution and interpolation methodology of the INCOIS LAS gridded ARGO product used as the PS's validation dataset; the full "Training Input Datasets" table referenced but not rendered on the SIH26066 portal mirror; full text and reported numeric results of the IEEE TGRS DNN paper and the 2026 graph-attention paper beyond their abstracts.

This document is a working v1 build bible, not a substitute for reading the primary sources it references. Treat every [D]-tagged claim as "verify before quoting a specific number to judges," and resolve the Part 1 dataset-table gap before finalizing your data pipeline plan.